

实践11：基于Project Instinct的G1跑酷策略训练与Sim2Sim验证



深蓝学院《人形机器人运动控制》课程第7章

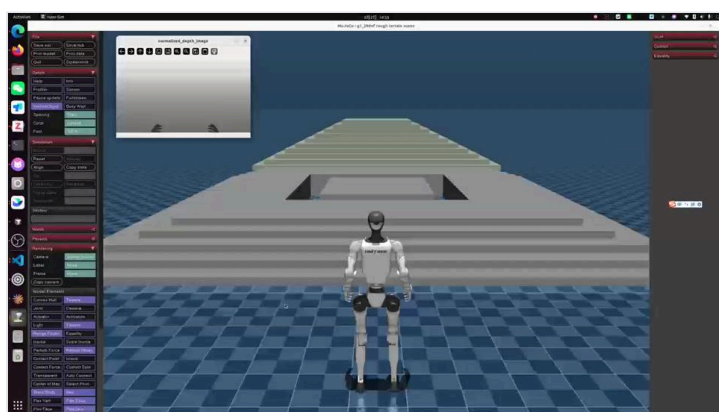
课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

本次作业的目标是在 `instinctlab` 项目中复现 Project Instinct 的 G1 Parkour 训练任务，完成 `Instinct-Parkour-Target-Amp-G1-v0` 的策略训练，并将训练好的策略导出后在 MuJoCo 中进行 Sim2Sim 验证。

完成本次作业后，应能够理解：

- InstinctLab 与 IsaacLab、Instinct-RL 的关系。
- 带动作参考数据的 AMP parkour 任务如何配置和启动训练。
- 训练得到的 G1 29DoF 策略如何导出为 ONNX 模型。
- Sim2Sim 中如何把 MuJoCo 状态、深度图观测和历史观测组织成 policy 输入。
- IsaacSim 训练策略迁移到 MuJoCo 时需要保持哪些接口一致。



2. 作业背景

前面章节中，我们已经训练过基于本体感知或地形高度扫描的 G1 行走策略。此类任务通常直接以速度跟踪、姿态稳定和能量约束等人为设计奖励作为主要学习目标。

Instinct 的 parkour 任务进一步引入动作参考数据和视觉感知。 `Instinct-Parkour-Target-Amp-G1-v0` 任务使用 G1 29DoF 机器人，在粗糙地形和目标引导环境中训练具备 parkour 能力的策略。策略输入不仅包含本体状态、速度命令、关节状态和上一帧动作，还包含深度相机观测经过 depth encoder 后得到的视觉特征。训练中还会使用动作参考数据，使机器人学习更接近参考运动的全身动作模式。

Project Instinct 的 Hiking in the Wild 页面提供了论文、代码入口以及 Data & Model 下载入口。本次作业使用该页面提供的动作参考数据，课程也已经将其作为 `hiking-in-the-wild_Data&Model.zip` 附件提供给大家。

项目页面：

- <https://project-instinct.github.io/hiking-in-the-wild/>

GitHub 开源代码入口：

- <https://github.com/project-instinct/instinctlab>：Project Instinct 的仿真环境、训练任务和配置代码，本作业的 parkour 任务位于该仓库中。
- https://github.com/project-instinct/instinct_rl：与 InstinctLab 配套的强化学习算法库，提供 PPO、WASABI/AMP、策略网络和训练 runner。
- <https://github.com/project-instinct>：Project Instinct 的 GitHub 组织主页，可查看训练、部署和动作处理等其他开源仓库。

参考文献：

- Shaoting Zhu, Ziwen Zhuang, Mengjie Zhao, Kun-Ying Lee, and Hang Zhao. “Hiking in the Wild: A Scalable Perceptive Parkour Framework for Humanoids.” arXiv preprint arXiv:2601.07718, 2026. 论文链接：<https://arxiv.org/abs/2601.07718>

3. 环境准备

3.1 安装 IsaacLab、Instinct-RL 和 instinctlab

如果在第2章实践2中已经完成 IsaacLab 和 conda 环境安装，可以继续使用实践2的 conda 环境进行本次训练。建议环境配置与 `instinctlab` 仓库要求保持一致：

代码块

```
1 IsaacSim 5.1.0
2 IsaacLab v2.3.2
3 Python 3.11
4 conda 环境名称可沿用第2章环境，例如 env_isaaclab
```

`instinctlab` 项目的安装方法不在本作业文档中重复展开。请直接参考仓库 [README](#)
`/path/to/instinctlab/README.md`

其中需要完成的核心安装内容包括：

- 安装与 IsaacSim 5.1.0 兼容的 IsaacLab。
- 安装 `instinct_rl`。
- 安装 `instinctlab`。
- 确认可以从 `instinctlab` 根目录运行训练脚本。

进入 conda 环境的示例命令如下：

代码块

```
1  conda activate env_isaacLab
2  cd /path/to/instinctlab
```

如果你的第2章环境名称不同，请替换为自己的 conda 环境名称。

3.2 作业代码目录

本次作业主要使用如下目录：

代码块

```
1  instinctlab/
2  |
3  |-- README.md
4  |
5  |-- scripts/
6  |   |-- instinct_rl/
7  |       |-- train.py
8  |
9  |-- source/instinctlab/
10 |   |-- instinctlab/
11 |       |-- tasks/
12 |           |-- parkour/
13 |               |-- README.md
14 |               |-- scripts/
15 |                   |-- play.py
16 |                   |-- config/
17 |                       |-- parkour_env_cfg.py
18 |                       |-- g1/
19 |                           |-- g1_parkour_target_amp_cfg.py
20 |                           |-- agents/
21 |                               |-- instinct_rl_amp_cfg.py
22 |
23 |-- hiking-in-the-wild_Data&Model/ #课程代码hiking-in-the-wild_Data&Model.zip提供
24 |   |-- data&model/
```

```

25 | | | |-- parkour_motion_reference/
26 | | | |-- parkour_motion_without_run_retargetted.npz
27 | | | |-- parkour_motion_without_run.yaml
28 |
29 |-- sim2sim/ #课程代码sim2sim.zip提供
30 | |-- config.py
31 | |-- mujoco_env.py
32 | |-- onnx_inference.py
33 | |-- sim2sim.py
34 | |-- g1_29dof/
35 | | |-- meshes_recv_1_0/
36 | | |-- g1_29dof_rev_1_0.xml
37 | | |-- scene_rough.xml
38 | | |-- scene_flat.xml
39 | | |-- rough_terrain_generator.py
40 | | |-- rough_terrain_cfg.py
41 | |-- policy/
42 | | |-- stand_actor.onnx
43 | | |-- stand_depth_encoder.onnx
44 | | |-- parkour_actor.onnx
45 | | |-- parkour_depth_encoder.onnx
46 |
47 instinct_rl/
48 |
49 |-- instinct_rl/
50 | |-- runners/
51 | | |-- on_policy_runner.py
52 | |-- algorithms/
53 | | |-- ppo.py
54 | | |-- wasabi.py
55 | |-- modules/
56 | | |-- actor_critic.py
57 | | |-- encoder_actor_critic.py
58 | | |-- moe_actor_critic.py
59 | | |-- moe.py
60 | | |-- all_mixer.py
61 | | |-- discriminator.py
62 | | |-- parallel_layer.py
63 | | |-- conv2d.py
64 | |-- storage/
65 | | |-- rollout_storage.py
66 | | |-- amp_storage.py
67 | |-- utils/
68 | | |-- utils.py

```

其中：

instinctlab 任务与仿真文件：

- `g1_parkour_target_amp_cfg.py`：G1 parkour AMP 训练任务配置，包含动作参考数据路径。
- `instinct_rl_amp_cfg.py`：指定本任务使用 `WasabiPPO`、`EncoderMoEActorCritic`、深度图编码器及相关训练超参数。
- `parkour_env_cfg.py`：定义 parkour 环境、奖励以及 `amp_policy`、`amp_reference` 等观测组。
- `parkour/README.md`：Instinct 官方给出的 parkour 任务使用说明。
- `train.py`：训练入口。
- `play.py`：策略回放和导出入口。
- `sim2sim/config.py`：MuJoCo 模型路径、ONNX policy 路径、关节顺序、PD 参数和观测配置。
- `sim2sim/mujoco_env.py`：MuJoCo 环境、viewer、传感器和深度相机封装。
- `sim2sim/onnx_inference.py`：ONNX Runtime 推理封装。
- `sim2sim/sim2sim.py`：本次 Sim2Sim 作业主程序，其中包含需要同学补全的 TODO。

instinct_rl 训练算法文件：

- `on_policy_runner.py`：执行采样、算法更新、日志记录和 checkpoint 保存的训练主循环。
- `ppo.py`：实现 PPO 的优势估计、裁剪目标、价值损失和参数更新。
- `wasabi.py`：在 PPO 上加入动作参考判别器奖励和梯度惩罚，是本任务使用的 AMP/WASABI 算法核心。
- `actor_critic.py`、`encoder_actor_critic.py`、`moe_actor_critic.py` 和 `all_mixer.py`：共同组成带观测编码器和 Mixture-of-Experts 的 Actor-Critic 策略。
- `moe.py`：实现 MoE 的门控网络和专家层。
- `discriminator.py`：区分策略动作状态与参考动作状态，并为 AMP 训练提供奖励。
- `parallel_layer.py` 和 `conv2d.py`：实现并行观测编码模块及深度图卷积编码器。
- `rollout_storage.py`：保存 PPO rollout，并生成训练所需的 mini-batch。
- `amp_storage.py`：保存策略状态和参考动作状态，供 AMP 判别器训练使用。
- `utils.py`：提供观测切片、拼接和子观测维度计算等工具。

4. 作业核心任务

- 训练任务 ID： `Instinct-Parkour-Target-Amp-G1-v0`
- 训练仓库： `/path/to/instinctlab`

- 动作参考数据目录: `/path/to/instinctlab/hiking-in-the-wild_Data&Model/data&model/parkour_motion_reference`
- 动作参考选择文件: `parkour_motion_without_run.yaml`
- 训练入口: `scripts/instinct_rl/train.py`
- 回放与导出入口:
`source/instinctlab/instinctlab/tasks/parkour/scripts/play.py`
- Sim2Sim 入口: `sim2sim/sim2sim.py`

本次作业需要完成:

- 准备 `hiking-in-the-wild_Data&Model` 动作参考数据。
- 确认 `AmassMotionCfg` 中的 `path` 和 `filtered_motion_selection_filepath` 指向正确位置。
- 启动 `Instinct-Parkour-Target-Amp-G1-v0` 训练。
- 使用 `play.py` 加载训练好的 checkpoint, 观察策略效果。
- 使用 `play.py --exportonnx --useonnx` 导出 ONNX 策略。
- 补全 `sim2sim/sim2sim.py` 中深度图处理相关 TODO。
- 在 `sim2sim/config.py` 中配置训练导出或附件提供的 ONNX 模型路径, 运行 MuJoCo Sim2Sim。

5. 数据准备

5.1 获取动作参考数据

动作参考数据可从 Project Instinct 的 Hiking in the Wild 页面中 `Data & Model` 下载, 也可以使用本次作业附件 `hiking-in-the-wild_Data&Model.zip`。

将压缩包解压到 `instinctlab` 根目录后, 应得到如下结构:

代码块

```
1 /path/to/instinctlab/hiking-in-the-
  wild_Data&Model/data&model/parkour_motion_reference/
2 |-- parkour_motion_without_run_retargetted.npz
3 |-- parkour_motion_without_run.yaml
```

如果使用命令行解压, 注意路径中包含 `&`, 需要加引号:

代码块

```
1 cd /path/to/instinctlab
```

```
2 unzip "/path/to/course-materials/hiking-in-the-wild_Data&Model.zip" -d .
```

5.2 配置动作参考路径

根据

`instinctlab/source/instinctlab/instinctlab/tasks/parkour/README.md`，需要修改

`source/instinctlab/instinctlab/tasks/parkour/config/g1/g1_parkour_target_amp_cfg.py` 文件

中的 `AmassMotionCfg` 设置：

代码块

```
1 class AmassMotionCfg(AmassMotionCfgBase):
2     path = "/path/to/instinctlab/hiking-in-the-
    wild_Data&Model/data&model/parkour_motion_reference"
3     retargeting_func = None
4     filtered_motion_selection_filepath = (
5         "/path/to/instinctlab/hiking-in-the-
    wild_Data&Model/data&model/parkour_motion_reference/" "parkour_motion_without_run.yaml"
6     )
```

这里两个字段的含义是：

- `path`：动作参考数据所在目录。`parkour_motion_without_run.yaml` 中列出的 `.npz` 文件会相对该目录查找。
- `filtered_motion_selection_filepath`：动作参考选择文件，用来指定本次训练使用哪些 motion 文件。

上述路径为示例路径，请将 `/path/to/instinctlab` 替换为本机的 `instinctlab` 仓库绝对路径。

6. 启动训练

6.1 基础训练命令

进入 conda 环境和 `instinctlab` 根目录：

代码块

```
1 conda activate env_isaacrlab
2 cd /path/to/instinctlab
```

启动训练：

代码块

```
1 python scripts/instinct_rl/train.py --headless --task=Instinct-Parkour-Target-Amp-G1-v0
```

如果显存不足，可以减少并行环境数量：

代码块

```
1 python scripts/instinct_rl/train.py --headless --task=Instinct-Parkour-Target-Amp-G1-v0 --num_envs 512
```

如果需要在 IsaacSim 中观察训练环境，可以去掉 `--headless`，但训练速度会明显降低：

代码块

```
1 python scripts/instinct_rl/train.py --task=Instinct-Parkour-Target-Amp-G1-v0 --num_envs 64
```

6.2 训练输出

训练运行后会在 logs 目录下生成 run。具体路径以终端输出为准，一般会包含任务名和时间戳。

记录你的训练 run 路径，后续 `play.py` 和导出 ONNX 时需要使用。例如：

代码块

```
1 /path/to/instinctlab/logs/instinct_rl/parkour_target_amp_g1/<run_name>
```

实际目录名称可能不同，应以你自己的训练输出为准。

可以使用 TensorBoard 查看奖励、损失和训练进度等曲线。在 `instinctlab` 仓库根目录运行：

代码块

```
1 tensorboard --logdir logs/instinct_rl --port 6006
```

然后在浏览器中打开：

代码块

```
1 http://localhost:6006
```

如果训练运行在远程服务器上，需要通过 SSH 端口转发或 IDE 的端口转发功能访问 `6006` 端口。

7. 回放与导出策略

7.1 回放训练结果

训练完成或训练到一定 checkpoint 后，使用 `play.py` 回放策略。回放应使用 `-Play-v0` 任务，它默认开启了速度指令可视化、关闭了边界墙，更适合观察策略效果：

代码块

```
1 cd /path/to/instinctlab
2 python source/instinctlab/instinctlab/tasks/parkour/scripts/play.py \
3     --task=Instinct-Parkour-Target-Amp-G1-Play-v0 \
4     --load_run=<run 目录名> \
5     --checkpoint=<checkpoint 文件名>
```

两个参数的含义：

- `--load_run`： `logs/instinct_rl/g1_parkour/` 下的 run 目录名，例如 `20260729_115301`，**不是完整的绝对路径**。
- `--checkpoint`：该 run 下的 checkpoint 文件名，例如 `model_15000.pt`。

7.2 导出 ONNX

将训练好的策略导出为 ONNX。导出应使用训练任务 `Instinct-Parkour-Target-Amp-G1-v0`（而非 `-Play-v0`），因为导出时环境需要与训练时的观测空间完全一致：

代码块

```
1 cd /path/to/instinctlab
2 python source/instinctlab/instinctlab/tasks/parkour/scripts/play.py \
3     --task=Instinct-Parkour-Target-Amp-G1-v0 \
4     --load_run=<run 目录名> \
5     --checkpoint=<checkpoint 文件名> \
6     --exportonnx \
7     --num_envs 1
```

参数说明：

- `--load_run`：与回放一致，填写 run 目录名（如 `20260729_115301`）。
- `--checkpoint`：要导出的 checkpoint 文件名（如 `model_15000.pt`）。

- `--exportonnx`：执行 ONNX 导出。
- `--num_envs 1`：ONNX 导出要求环境数为 1。

导出完成后，找到导出目录中的 `actor.onnx` 和 `0-depth_encoder.onnx`，然后在 `sim2sim/config.py` 中将 parkour 模型路径改为对应文件的实际路径：

代码块

```
1  PARKOUR_POLICY_FILE = "/path/to/exported/actor.onnx"
2  PARKOUR_DEPTH_ENCODER_FILE = "/path/to/exported/0-depth_encoder.onnx"
```

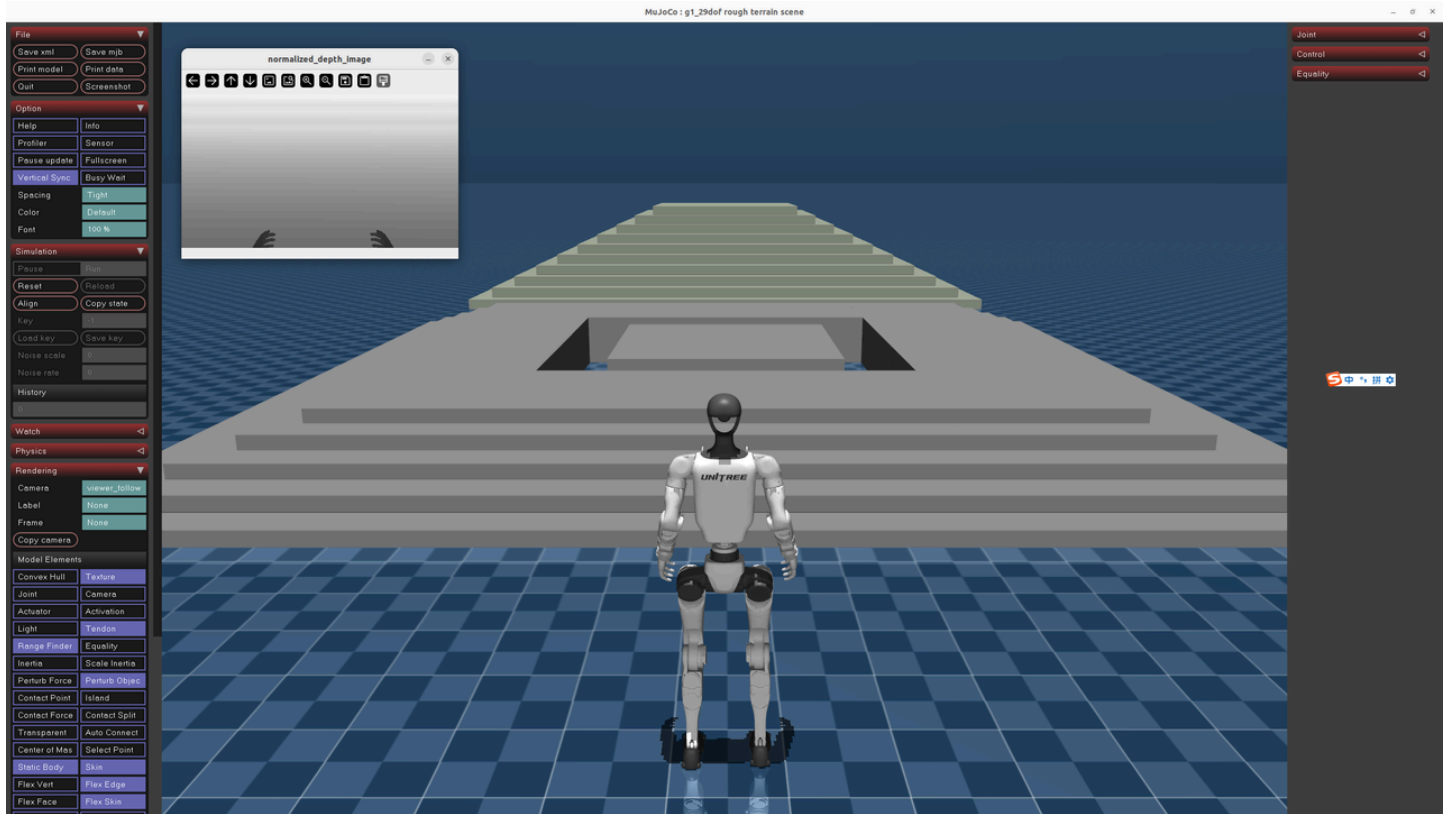
如果暂时没有完成完整训练，也可以先使用课程提供的示例 ONNX 文件验证 Sim2Sim 主流程。

8. Sim2Sim

8.1 Sim2Sim 目标

本次 Sim2Sim 的目标是把 IsaacSim 中训练得到的 G1 parkour policy 放到 MuJoCo 中运行。由于两个仿真器的模型、传感器和数据接口不同，Sim2Sim 代码需要显式处理以下一致性问题：

- 关节顺序一致：MuJoCo/SDK 顺序和 policy/IsaacSim 顺序不同，需要用索引映射。
- 关节符号一致：部分关节在 MuJoCo 模型和训练模型中的正方向可能不同，需要用 `JOINT_SIGNS` 修正。
- 默认关节角一致：policy 输出的是相对默认姿态的动作，需要加上 `DEFAULT_JOINT_POS`。
- 动作尺度一致：policy 输出需要乘以 `ACTION_SCALES`。
- PD 控制参数一致：MuJoCo 中需要使用训练或部署配置对应的 stiffness、damping 和 torque limit。
- 观测顺序一致：本体观测、历史观测和深度图特征必须按照训练时的顺序拼接。
- 深度图预处理一致：resize、crop、inpaint、blur、clip 和归一化必须与训练或部署时保持一致。



8.2 安装 Sim2Sim 依赖

可以继续使用训练时的 conda 环境。激活环境并进入仓库：

代码块

```
1 conda activate env_isaacrlab
2 cd /path/to/instinctlab
```

前面执行 `python -m pip install -e source/instinctlab` 时已经安装了 NumPy、OpenCV 等项目依赖。**如果**当前环境尚未安装 MuJoCo，执行：

代码块

```
1 python -m pip install mujoco
```

如果已经安装过 MuJoCo，可以跳过这一步。然后安装 ONNX Runtime：

代码块

```
1 python -m pip install onnxruntime
```

使用以下命令检查依赖是否安装成功：

代码块

```
1 python -c "import mujoco, onnxruntime; print('MuJoCo:', mujoco.__version__)"
```

8.3 Sim2Sim 代码结构与关键实现说明

`sim2sim/sim2sim.py` 中主要类如下：

- `RingBuffer`：保存本体观测历史。
- `DepthImagePipeline`：保存深度图历史，并完成深度图预处理。
- `Sim2simInstance`：封装 MuJoCo 环境、ONNX policy、观测构造、动作解码和 PD 控制。

从执行流程上看，`sim2sim.py` 可以概括为四步：观测获取、策略推理、动作解码、环境推进。代码主循环正是按这个顺序运行：先更新观测，再调用 ONNX policy 推理，接着将策略输出转换为 MuJoCo 的控制量，最后通过多次物理步推进仿真状态。

这四步构成了完整的 RL 闭环（observation → action → next_observation），总结如下：

① 观测获取	读 sensordata，计算 projected_gravity、joint_pos_rel 等，写入历史 buffer	构造好的 observation vector
② 策略推理	depth encoder 编码深度图特征 + actor 输出 raw_action	29 维归一化动作
③ 动作输出	关节顺序映射 × action_scale + default_joint_pos × joint_signs，再经 PD 控制生成力矩	各关节目标力矩 τ
④ 环境推进	mj_step() × DECIMATION 次，更新 qpos、qvel、sensordata	下一帧的仿真状态（回到步骤 ①）

8.3.1 第一步：观测获取

`Sim2simInstance.update_observation()` 中构造的本体观测包括：

代码块

```
1 base_angle_vel
2 projected_gravity
3 velocity_commands
4 joint_pos_rel
5 joint_vel_rel
6 last_action
```

每个观测项会写入历史 buffer，随后在 `get_history_obs()` 中展开并拼接。本体观测之外，深度图也会维护独立的历史队列，并在后续送入 depth encoder。

在这一步中，需要特别注意以下几个关键实现点。

`projected_gravity` 是世界系重力方向投影到机身坐标系的结果。Sim2Sim 中通过 IMU 四元数和 `quat_apply_inverse()` 得到：

代码块

```
1 projected_gravity = quat_apply_inverse(  
2     root_quat, np.array([0, 0, -1], dtype=np.float32)  
3 )
```

这里需要注意四元数格式。当前代码注释说明使用的是 `wxyz` 格式。

`last_action` 必须使用 policy 顺序保存，并在下一帧作为观测输入。它表示上一时刻 actor 输出的原始动作，用来帮助策略感知自身刚刚做过什么，从而保持动作连续和平滑。

`DepthImagePipeline` 中不是直接使用最新一帧深度图，而是维护一个长度为 60 的队列，并从中抽取 8 帧作为输入：

代码块

```
1 self.indices = np.linspace(  
2     -1 - down_sample_factor * (self.nb_frames - 1), -1, self.nb_frames  
3 ).astype(np.int32)
```

训练侧在 `parkour_env_cfg.py` 中配置了 `delayed_frame_ranges=(0, 1)`，即每次获取深度图观测时，在 `[0, 1]` 帧范围内随机延迟，模拟真实传感器延迟。Sim2Sim 中

`DepthImagePipeline` 通过以下方式实现相同效果：

- 每帧 `append()` 处理完后，调用 `resample_delay()` 在 `delay_ranges` 范围内随机采样一个延迟值 `self.delay`。
- `get_depth_obs()` 从历史队列取帧时，将索引减去 `self.delay`，即取更早的历史帧，模拟延迟效果。

对应参数 `delay_ranges` 默认为 `(0, 1)`，与训练侧一致；设为 `(0, 0)` 可关闭延迟。

8.3.2 第二步：策略推理

`get_history_obs()` 会先将各个本体观测历史展开并拼接，再将深度图历史送入 depth encoder 提取视觉特征，最后把两部分拼接成 actor 的输入。也就是说，actor 并不是直接接收原始深度图，而是接收“本体历史观测 + 深度编码特征”的组合输入。

这一阶段可以概括为：

- `RingBuffer` 提供最近若干帧的本体观测历史。
- `DepthImagePipeline` 提供经过预处理、带有历史和延迟效果的深度图输入。

- `depth_encoder` 将深度图历史编码为视觉特征。
- `actor` 根据完整历史观测输出当前时刻动作 `raw_action`。

8.3.3 第三步：动作输出

策略输出的 `raw_action` 仍然是 policy 顺序下的归一化动作，并不能直接作为 MuJoCo 控制输入。Sim2Sim 需要先把它变换回 robot 顺序，再结合动作缩放、默认关节位置和 PD 控制生成最终力矩命令。

首先需要处理关节顺序映射。

`config.py` 中同时定义了：

代码块

```
1 UNITREE_G1_29DOF_SDK_JOINT_NAMES
2 POLICY_JOINT_NAMES
```

MuJoCo 传感器读出的关节顺序使用 SDK/robot 顺序，而 policy 输入和输出使用 IsaacSim 训练顺序。因此代码中需要两组映射：

代码块

```
1 self.robot_to_policy = self.compute_joint_indices(robot_joint_name,
    policy_joint_name)
2 self.policy_to_robot = self.compute_joint_indices(policy_joint_name,
    robot_joint_name)
```

使用方式：

- 构造 `joint_pos_rel` 和 `joint_vel_rel` 时，从 robot 顺序转成 policy 顺序。
- 将 policy 输出动作下发到 MuJoCo 前，从 policy 顺序转回 robot 顺序。

在完成顺序变换后，策略输出会先经过 `action_scale` 和 `default_joint_pos` 还原为目标关节位置，再结合 `joint_signs` 映射到 MuJoCo 侧的关节定义。随后，Sim2Sim 不直接输出目标位置，而是通过 PD 控制生成力矩：

- 比例项根据目标位置和当前关节位置的误差计算。
- 微分项根据当前关节速度提供阻尼。
- 最终力矩经过 `torque_limit` 截断后写入 `self.sim_env.model_data.ctrl[:]`。

此外，代码中还使用 `DECIMATION`，即一次策略输出对应多个 MuJoCo 物理步。这使得策略控制频率与底层仿真积分频率解耦，更接近训练时的控制设置。

8.3.4 第四步：环境推进

当第三步已经得到当前控制周期的关节力矩 `tau` 后，Sim2Sim 并不会立刻回到策略网络重新推理，而是先让 MuJoCo 在这组控制输入下连续推进若干个物理步：

代码块

```
1  for _ in range(decimation):
2      tau = (
3          self.stiffness * (action - self.sim_env.model_data.sensordata[:29])
4          - self.damping * self.sim_env.model_data.sensordata[29:58]
5      )
6      tau = np.clip(tau, a_min=-self.torque_limit, a_max=self.torque_limit)
7      self.sim_env.model_data.ctrl[:] = tau
8      self.sim_env.physical_step()
```

这里的 `self.sim_env.physical_step()` 可以理解为执行一次 MuJoCo 的底层积分更新。每调用一次，仿真器都会根据当前关节力矩、接触、重力和系统动力学，更新机器人状态，包括：

- 关节位置 `qpos`
- 关节速度 `qvel`
- IMU、关节传感器等对应的 `sensordata`
- 刚体位姿、碰撞接触和下一时刻的动力学状态

因此，MuJoCo 中“环境如何变化”并不是额外写一套状态转移逻辑，而是通过反复调用物理步函数，让仿真器自己根据动力学方程推进系统。这正对应 RL 闭环中的状态转移过程：

代码块

```
1  observation_t -> action_t -> DECIMATION x MuJoCo physical_step() ->
   observation_{t+1}
```

第四步的作用，就是把策略给出的动作真正施加到仿真环境中，并得到下一时刻的状态。

这里的 `DECIMATION` 表示：策略每输出一次动作，不是只执行一个 MuJoCo 物理步，而是会连续执行多个物理步。比如当 `SIM_DT = 0.005`、`DECIMATION = 4` 时，MuJoCo 每个物理步前进 `0.005 s`，而同一个动作会连续执行 4 个物理步，也就是总共推进 `0.02 s`。因此，底层物理仿真频率是 `200 Hz`，策略控制频率是 `50 Hz`。

这样设计的好处是：

- 物理积分可以保持较高频率，接触和动力学更稳定。
- 策略不需要每个最小积分步都重新推理，计算量更低。
- 控制频率更容易对齐训练环境中的设置，减少 sim-to-sim 偏差。

完成这若干个物理步之后，代码会调用 `viewer.sync()` 刷新可视化，并通过：

```
代码块time_to_sleep = SIM_DT * DECIMATION - (time.perf_counter() - time_start)
```

尽量让每个控制周期的墙钟时间接近 `SIM_DT * DECIMATION`。随后程序回到第一步，重新读取新的传感器数据，构造下一帧观测，形成完整闭环。

8.4 需要补全的内容

本次作业要求补全：

代码块

```
1 /path/to/instinctlab/sim2sim/sim2sim.py
```

重点补全 `DepthImagePipeline.append()` 中的深度图预处理流程。同学需要根据下面的功能要求、`config.py` 中的参数，以及 OpenCV API 自行补全。

`DepthImagePipeline.append()` 按照 `resize` → `crop` → `inpaint` → `Gaussian blur` → `clip` → `normalize` → `append` 的顺序处理深度图，需要补全以下内容：

1. 调整图像尺寸

- `self.shape` 按 `(height, width)` 保存，因此先解包为 `h, w`。
- 调用 `cv2.resize` 时，`dsize` 的顺序是 `(width, height)`，即 `(w, h)`。
- 插值方式 `interpolation` 为 `cv2.INTER_NEAREST`（最近邻插值）。
- `resize` 后 NumPy 图像的形状应为 `(h, w)`。

2. 裁剪图像

- `self.crop_region` 的顺序为 `(y1, y2, x1, x2)`，分别表示从上、下、左、右裁掉的像素数。
- NumPy 二维图像按 `[行, 列]` 索引，因此高度方向使用 `y1 : h - y2`，宽度方向使用 `x1 : w - x2`。

3. 修补无效深度

- 使用 `(image < 0.2)` 找出过近或无效的深度像素，并转换为 `np.uint8 mask`。
- 调用 `cv2.inpaint` 时传入深度图、`mask`、修补半径 `3` 和算法 `cv2.INPAINT_NS`。
- `mask` 中非零位置是需要根据周围有效像素进行修补的区域。

4. 平滑深度图

- 使用 `cv2.GaussianBlur` 降低深度噪声。
- 高斯核大小设为 `(3, 3)`，`sigmaX=1`，`sigmaY=1`。
- 处理后的图像尺寸和数据类型应与输入保持一致。

5. 裁剪并归一化深度值

- 使用 `np.clip` 将深度限制在 `[self.near_clip, self.far_clip]`。
- 使用线性归一化将深度映射到 `[0, 1]`：

代码块

```
1 normalized = (image - near_clip) / (far_clip - near_clip)
```

6. 写入历史队列

- 将处理后的 `np.float32` 深度图写入 `self.que`，供 `get_depth_obs()` 按时间索引抽取。



相关 OpenCV 官方教程：

- `cv2.resize`：Image Resizing
- `cv2.inpaint`：Image Inpainting
- `cv2.GaussianBlur`：Smoothing Images

补全后，`DepthImagePipeline.append()` 的输出尺寸应与 `self.obs_depth_image_shape` 一致。根据当前配置：

代码块

```
1 RESIZED_DEPTH_IMAGE_SHAPE = (36, 64) # (高度, 宽度)
2 OBS_DEPTH_IMAGE_CROP_REGION = (18, 0, 16, 16) # (上, 下, 左, 右)
```

裁剪后的单帧深度图尺寸为：

代码块

```
1 (36 - 18 - 0) x (64 - 16 - 16) = 18 x 32
```

depth pipeline 会从历史图像中采样 `nb_frames=8` 帧，因此送入 depth encoder 的形状应为：

代码块

```
1 (1, 8, 18, 32)
```

8.5 运行 Sim2Sim

进入 Sim2Sim 目录：

代码块

```
1 cd /path/to/instinctlab/sim2sim
```

运行 parkour policy：

代码块

```
1 python sim2sim.py --task parkour
```

运行站立 policy：

代码块

```
1 python sim2sim.py --task stand
```

parkour 模式支持方向键或数字小键盘增量调整速度命令：

代码块

```
1 ↑ / 8：增加前向速度
2 ↓ / 2：减小前向速度
3 ← / 4：增加左转角速度
4 → / 6：增加右转角速度
5 空格 / 5：速度命令归零
6 R：重置机器人、速度命令、上一帧动作和观测历史
```

每次按键分别按 `COMMAND_STEP` 增减速度，并使用 `COMMAND_RANGES` 限制命令范围。如果数字小键盘没有响应，请确认键盘 NumLock 状态，也可以直接使用方向键。

9. 提交要求及评分标准

提交内容包括：

- 补全后的 `sim2sim/sim2sim.py`。
- MuJoCo Sim2Sim 运行截图或视频。
- 简短实验报告，说明训练是否成功、Sim2Sim 是否稳定，以及遇到的问题和解决方式。

总分 100 分：

--	--	--

项目	分值	要求
补全后的 sim2sim/sim2sim.py	25	正确补全 DepthImagePipeline.append() 中的深度图预处理流程；能够完成 resize、crop、无效深度 mask、cv2.inpaint 修补、Gaussian blur、clip 和 [0, 1] 归一化；处理后的图像尺寸、数据类型和 depth encoder 输入要求一致；运行 python sim2sim.py --task parkour 时程序能够正常加载 ONNX 并进入 MuJoCo 仿真
MuJoCo Sim2Sim 运行视频	25	提交 MuJoCo Sim2Sim 运行视频；画面中应能看到 G1 机器人和仿真环境；视频或截图能够体现使用 parkour policy 控制，而不是仅加载静止模型；如提交视频，建议展示机器人在 parkour 模式下的运动或键盘速度命令响应
简短实验报告	50	说明训练是否成功；说明 Sim2Sim 是否稳定，以及机器人是否存在抖动、摔倒、偏转或无法响应命令等现象；解释深度图预处理和历史观测的作用；记录遇到的问题和解决方式

10. 常见问题 Q&A

10.1 训练时报 No motion files found

优先检查：

- `path` 是否指向 `parkour_motion_reference` 目录。
- `filtered_motion_selection_filepath` 是否指向 `parkour_motion_without_run.yaml` 文件。
- `.yaml` 中列出的 `.npz` 文件是否真实存在于 `path` 目录下。
- 路径中 `hiking-in-the-wild_Data&Model` 和 `data&model` 是否拼写一致。

10.2 Sim2Sim 无法加载 ONNX

检查：

- `sim2sim/config.py` 中的 `PARKOUR_POLICY_FILE` 是否指向实际存在的 actor ONNX 文件。
- `sim2sim/config.py` 中的 `PARKOUR_DEPTH_ENCODER_FILE` 是否指向实际存在的 depth encoder ONNX 文件。
- 是否安装了 `onnxruntime`。如果未安装，使用以下命令安装：

代码块

```
1 pip install onnxruntime
```

- 该命令会同时安装依赖 `flatbuffers`（`onnxruntime` 的序列化依赖）。

- actor 和 depth encoder 是否来自同一次训练或兼容配置。

10.3 Sim2Sim 机器人动作剧烈抖动

优先检查：

- 关节顺序映射是否正确。
- `JOINT_SIGNS` 是否被误改。
- `DEFAULT_JOINT_POS`、`ACTION_SCALES`、`STIFFNESS`、`DAMPING` 是否与导出策略匹配。
- 深度图 resize、crop、归一化是否正确。
- `SIM_DT` 和 `DECIMATION` 是否与策略控制频率一致。

10.4 Sim2Sim 深度图可视化窗口`cv2.imshow()`打开报错

训练使用的 conda 环境在激活时会把 Isaac Sim 的预打包依赖目录加入 `PYTHONPATH`，其中可能包含不支持窗口显示的 `opencv-python-headless`，并优先于环境中安装的 GUI 版 `opencv-python` 被导入。命令前添加 `PYTHONPATH=""` 会仅对本次运行临时清空该搜索路径，使 Sim2Sim 使用当前 conda 环境 `site-packages` 中的依赖，从而允许 `cv2.imshow()` 显示深度图窗口。该写法不会永久修改环境变量。如果使用不包含 Isaac Sim 路径的独立 Sim2Sim 环境，并且确认导入的是支持 GUI 的 OpenCV，则可以省略 `PYTHONPATH=""`。

运行 parkour policy：

代码块

```
1 PYTHONPATH="" python sim2sim.py --task parkour
```